Qwen3-VL image+text profiling

SGLang issue #4 — round-3 execution report

Bowen Wang · 06 September 2026 · Qwen3-VL-8B-Instruct on one NVIDIA H200

Summary

Issue #4 asks whether Qwen3-VL image workloads behave differently from the text-only case settled in issue #2, and specifically whether a piecewise CUDA graph *helps more* there. Both halves of that expectation turn out to need correcting, and a larger effect than either was found along the way.

- **The multimodal feature transport dominates.** Switching `--mm-feature-transport` from the current upstream default (`cpu`) to `cuda_ipc` cuts TTFT by **28.2%** with the prefill graph held off on both sides. Nothing about CUDA graphs is involved.
- **SGLang is faster than vLLM on the image path** — by 36.9% at the configuration issue #4 specifies — reversing #2's text-only result where SGLang was 67% slower. The gap is entirely on time-to-first-token; vLLM decodes about 9% faster.
- **The prefill CUDA graph does pay on image+text, but only for small images.** It is worth 16.3% at a 256×256 image and 14.0% at 360p, and is unmeasurable at 720p and above. What the graph recovers in *milliseconds* is set by the prefill token count and is nearly independent of whether those tokens are visual or textual; the image-to-text ratio changes how large that recovery looks as a *percentage*, because an image adds vision-encoder time to the denominator that no graph can touch (§7).
- **The piecewise backend could not be measured honestly.** On current upstream it silently falls back to eager execution for 92% of its graph-eligible calls while every configuration check still reports success.

Two candidate explanations for the graph result were tested against the data and discarded; the surviving one is labelled as a hypothesis, and its own quantitative predictions failed twice. Section 6 records that in full.

1 · What was measured, and on what

One stack for every arm, frozen before the first run. SGLang is built from source at `upstream/main ff1285cc28` merged with PR #33726, which adds Qwen3-VL to the breakable-CUDA-graph allowlist. Without that PR the breakable arms run a known DeepStack replay bug: they would produce plausible latencies attached to numerically wrong output, so a pre-merge pin could not have answered the graph question at all.

Model	Qwen/Qwen3-VL-8B-Instruct, revision 0c351dd01e (the revision the round-2 protocol pinned)
Feature check	deepstack_visual_indexes = [8, 16, 24], out_hidden_size = 4096 ⇒ replay width 12288 — the target genuinely exercises the path under test
Hardware	1 × NVIDIA H200, driver 595.71.05, CUDA 13.0
vLLM anchor	0.21.0, same torch build, prefix caching disabled to match
Known confound	torch 2.11 against upstream's pinned 2.13. Every arm shares it identically, so internal contrasts hold; absolute latencies are not production claims.

Table 1 — frozen stack. Full manifest in the repository.

How each arm was verified

Every lever in this experiment has a silent-degradation path: the transport falls back to CPU per tensor when its pool fills, the piecewise backend falls back to eager per subgraph, and deprecated flags are still accepted while meaning something else. None of these crash. So no number here is quoted unless the arm proved it ran the configuration it was asked for, on three independent kinds of evidence:

- the **resolved configuration** reported by the server, compared against what was requested;
- the **capture-time** log line naming the backend actually captured;

- **behaviour** — the fraction of benchmark prefill batches that actually ran under a graph, plus explicit scans for every known degradation signal.

An arm failing any of these is reported as **unverified** and excluded from every comparison, with its exact failure recorded. This is not ceremony: section 6 describes an arm that passed the first two checks, reported 100% of its prefill batches under a graph, produced the tightest variance in the whole study — and was executing eagerly 92% of the time.

2 · The two levers at a fixed workload

First bracket: one 720p image plus ~128 text tokens, concurrency 1, 400 prompts × 5 repetitions per arm, run in a fixed order with the baseline re-run at the end. Baseline drift over the whole bracket was **0.87%**, well inside the 5% gate.

prefill graph	cpu transport	cuda_ipc transport	transport effect
disabled	142.30 ms	102.17 ms	**-28.20%**
breakable	146.91 ms	105.91 ms	**-27.91%**
graph effect	**+3.24%**	**+3.66%**	

Table 2 — TTFT p50, median of 5 repetitions. Every cell verified; per-arm variation 1.4–1.8%.

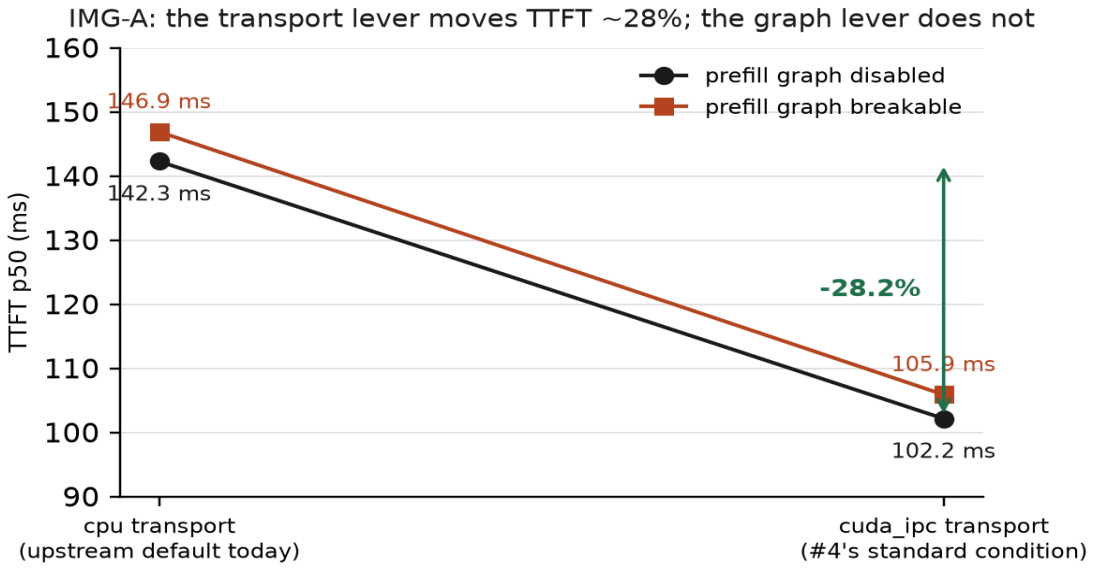

Figure 1 — the same four cells. The two lines are nearly parallel: each lever does the same thing regardless of the other's setting.

The interaction term is **+0.42 percentage points**, so the levers are independent. The transport is worth about 28% either way; the graph costs about 3–4% either way, which is below the 5% bar this study uses for a material difference and is reported as no material difference — though the sign is the same in three independent estimates.

Read against issue #4's own framing this matters twice over. #4 names IPC-on as the standard condition for SGLang image runs and the without-IPC case as the optional ablation. The cell it calls the baseline is therefore the 102.2 ms one — and a user who sets nothing on current upstream gets the 142.3 ms one instead. **The default costs a Qwen3-VL image deployment roughly 28% of its time-to-first-token, and no CUDA-graph setting recovers it.**

Against vLLM

comparison	SGLang	vLLM	gap
issue #2, text-only Case A	21.94 ms	13.12 ms	**SGLang +67%**
this study, #4's standard condition	102.17 ms	161.99 ms	**SGLang -36.9%**
this study, upstream default transport	146.91 ms	161.99 ms	**SGLang -9.3%**

Table 3 — the cross-framework picture inverts between the text and image paths.

The sign flip is the headline, and it is confined to first-token latency: per-output-token time is 5.68 ms for SGLang against 5.16 ms for vLLM, so vLLM decodes about 9% faster. A generation-heavy workload would rank these differently and nothing here speaks to that. The comparison is also an anchor rather than an equivalence: on image prompts the two frameworks produce the same content in different words, a divergence that phase-1 parity localised to the vision path, since both text fixtures matched token for token across all arms.

3 · Where the prefill graph does pay

One operating point cannot answer whether the graph ever helps on images. The second bracket sweeps nine workloads spanning prefill lengths from 128 to 2184 tokens, holding the transport at `cuda_ipc` throughout so the graph backend is the only variable, and running the two arms of each workload back to back so every comparison is its own bracket.

work load	image	visual tok	text tok	N	graph off	graph on	saving	effect	verdict
R0	none	0	128	128	27.5 ms	15.2 ms	+12.3 ms	**-44.8%**	graph wins
R1	256×256	66	142	208	55.2 ms	46.2 ms	+9.0 ms	**-16.3%**	graph wins
R2	360p	222	142	364	64.0 ms	55.0 ms	+9.0 ms	**-14.0%**	graph wins
R6	640×640	402	142	544	70.0 ms	66.8 ms	+3.2 ms	**-4.5%**	no material diff.
R7	768×768	578	142	720	78.7 ms	79.7 ms	-1.0 ms	**+1.3%**	not resolvable
R8	896×896	786	142	928	101.1 ms	99.7 ms	+1.4 ms	**-1.4%**	not resolvable
R3	720p	882	142	1024	104.5 ms	105.3 ms	-0.8 ms	**+0.8%**	not resolvable
R4	720p	882	1087	1969	135.7 ms	136.0 ms	-0.3 ms	**+0.2%**	not resolvable
R5	1080p	2042	142	2184	203.9 ms	206.0 ms	-2.1 ms	**+1.0%**	not resolvable

Table 4 — every cell independently verified; 300 prompts × 3 repetitions. R4 is the same 720p image with the text grown to ~1087 tokens. Resolution floor 3.60%, from repeating the reference cell.

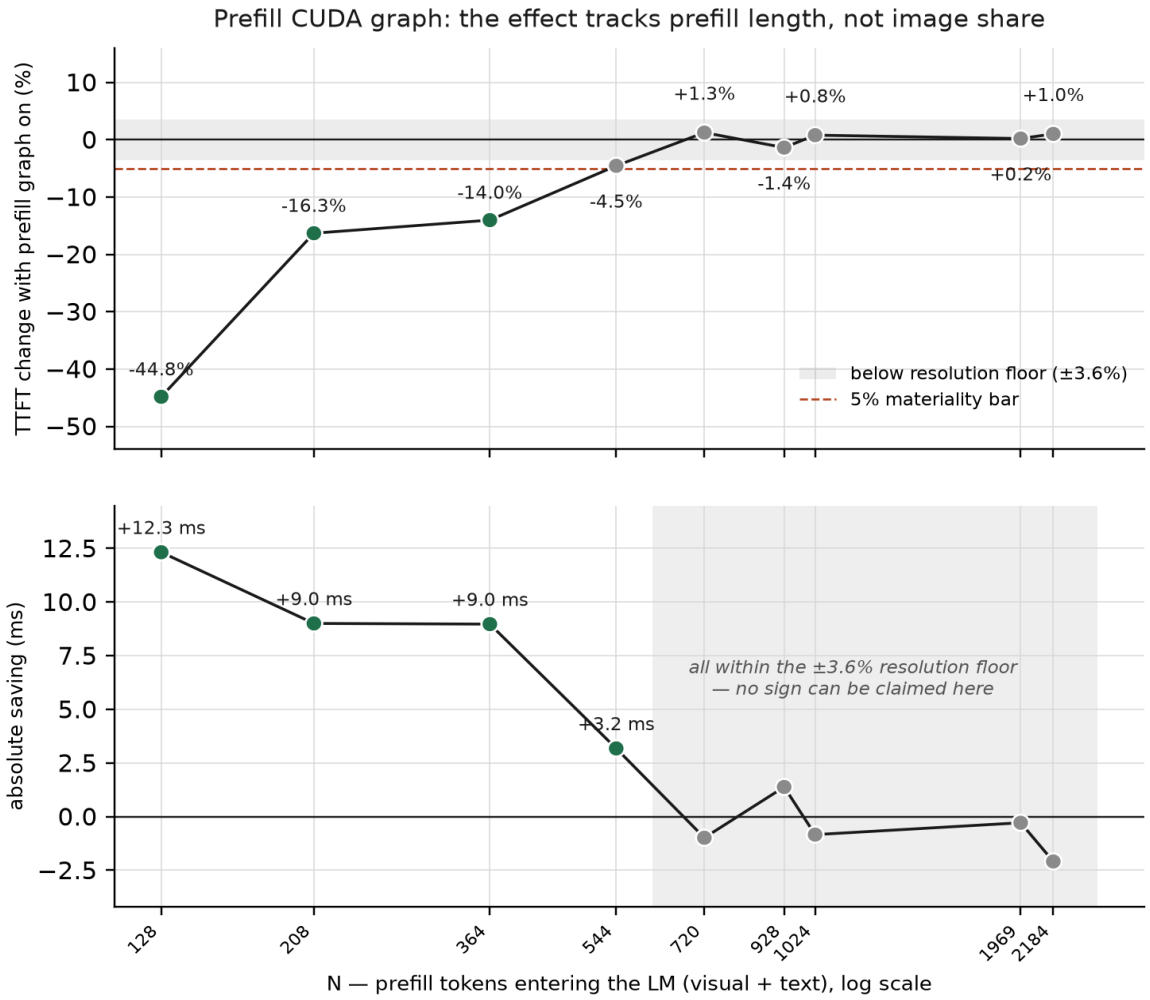

Figure 2 — upper panel: relative effect, with the shaded band marking what this bracket cannot resolve. Lower panel: the same result in milliseconds, where the behaviour is simpler — a saving that decays to nothing.

What the sweep says

The graph pays when the language-model prefill is short, and images are what makes it long. Qwen3-VL spends one visual token per 32×32 pixels, so the practical rule is about the image's token count rather than its file size.

regime	prefill tokens	recommendation
text-only, short prompts	~128	**enable the prefill graph** — 45% of TTFT
small image (≤ ~360p) + short text	208–364	**enable it** — 14–16% of TTFT
~640×640 + short text	544	marginal: 4.5%, resolvable but under the 5% bar
720p and above, any text length	≥ 720	no measurable effect either way

Table 5 — the operating guidance this study supports.

Under roughly 250 visual tokens the graph is clearly worth enabling; past about 600 it stops mattering. The material-win boundary falls between N = 364 and N = 544.

4 · The arm that could not be measured

Issue #4's hypothesis names the piecewise backend specifically, so its absence here is a real gap rather than a negative result. On current upstream the piecewise arm reported every sign of health: the resolved configuration said piecewise, the capture log said piecewise, all 2037 benchmark prefill batches reported running under a CUDA graph, and it produced the *lowest* variance of any arm in the study at 0.7%.

```
It was executing eagerly for 92.11% of its graph-eligible calls — 75200 of 81638, across two
shapes.
```

Two properties combine to hide this. The fallback is announced through a warn-once helper cached on the message string, so the log reports it exactly once however often it fires. And the fallback path returns *without* capturing, so a shape that once missed the capture stream stays eager for the life of the process. Onset is deterministic — the first fallback landed on graph-eligible call 6402 in two independent runs — which means a short run under-reports it badly: a 20-request smoke ended shortly after onset and scored 8.53%, while the full benchmark scored 92.11%.

The measurement exists only because counters were added to the pinned stack for this purpose. They are measurement-only and are not proposed upstream; the finding itself is written up separately for a new upstream issue rather than folded into #4. **The practical lesson is that low variance is not evidence of health.** An arm that is consistently eager is consistently eager, and ranking these arms by variance would have picked the broken one as the most trustworthy.

5 · Answers to issue #4

#4 asked	answer
Do image workloads behave differently from text-only?	**Yes, and in the opposite direction to the one assumed.** SGLang beats vLLM on the image path after trailing it by 67% on text.
Does the piecewise graph help more on images?	**No.** Graphs help *less* as images grow, because images add TTFT that no graph covers — vision encoding and preprocessing. Adding an 80-token image to a text prompt adds 28 ms.
Separate the IPC benefit from the graph benefit.	**Done, in a full 2×2.** Transport –28.2%, graph +3.7%, interaction +0.4 pp — independent levers.
Is there an image+text mix where the graph pays?	**Yes.** 16.3% at 256×256 and 14.0% at 360p with short text; nothing measurable at 720p and above.
Does TTFT include preprocessing and vision encoding?	**Yes**, and it dominates as images grow — which is precisely why the graph lever fades.

Table 6 — issue #4's questions against what was measured.

Recommendations

- **Set the transport explicitly.** On this workload `--mm-feature-transport=cuda_ipc` is worth 28% of TTFT, and it is not the default.
- **Enable the prefill graph for small-image workloads** — under roughly 250 visual tokens — and do not expect anything from it at 720p and above.
- **Do not rank configurations by variance.** The most degraded arm in this study had the tightest variance.
- **Report the piecewise fallback upstream as its own issue.** A warn-once that hides a 92% degradation is a measurement hazard for anyone benchmarking that backend.

6 · What went wrong, and what it cost

Three claims in this study were made and then withdrawn. They are recorded because the corrections shaped the design, and because a reader should be able to see which conclusions were arrived at rather than assumed.

claim	refuted by	outcome
The graph's benefit is a constant minus a per-token replay cost ($C - kN$).	R2: prefill length nearly doubled from 208 to 364 tokens and the saving did not move (9.00 → 8.97 ms).	withdrawn
The benefit tracks the image-to-text ratio.	R4: text share went from 13% to 55% at a fixed image and nothing changed. The same cell also rules out a pure length story at the top end.	withdrawn
Predicted values from the surviving hypothesis.	R5 missed three stated bands by a hair; R6 was predicted at 4–8 ms and came in at 3.18 ms, because the true decay is convex and the prediction interpolated linearly.	no longer used for prediction

Table 7 — discarded claims and the measurement that killed each.

The surviving explanation is that a CUDA graph recovers only the kernel-launch overhead not already hidden behind GPU execution. As each kernel is given more work the launch cost overlaps away and there is less to recover, so the saving decays toward zero instead of turning sharply negative. It accounts for the asymptote, for the step when an image first appears, and for R4's null result — but it is a hypothesis whose numeric predictions have failed twice, and it is not used to predict anything here.

7 · Follow-on: text tokens versus visual tokens

The sweep in section 3 moved visual tokens across seven points but moved text tokens only once, at 720p, where every cell sat inside the resolution floor. So the claim that composition does not matter had never been tested where a difference could show. Three text-only workloads were run whose token counts match image workloads already measured, making half of each comparison free.

prefill tokens	text-only saving	image saving	text-only effect	image effect
208	**+10.86 ms**	+9.00 ms	**−39.77%**	−16.30%
544	**+4.33 ms**	+3.18 ms	**−15.94%**	−4.54%
1024	**+0.45 ms**	−0.84 ms	−1.30%	+0.80%

Table 8 — matched token counts, different composition. Paired A/B/A/B blocking, three blocks per cell.

The saving column matches; the effect column does not. Across a 5× range the absolute saving differs by a roughly constant 1.2–1.9 ms while the percentage differs by a factor of 2–3.5. The same ~10 ms of recovered work reads as −39.8% on a text prompt and −16.3% on an image, because at this size the image carries **23.6 ms** of preprocessing and vision-encoder time underneath it. Both compositions still cross zero between 544 and 1024 tokens, so section 3's operating guidance is unchanged.

Two corrections earned here

- **A gate that read the wrong quantity.** The first pass discarded all three workloads on a drift gate applied to absolute latencies. But the 208-token cell's levels fall 19.4% across the bracket on a cold-start ramp while its paired effects agree to 2.98 pp — the drift that A/B/A/B blocking exists to cancel, cancelling as designed. Gating the levels put it straight back in.
- **A 24% padding artifact that looked like a result.** A 1024-token text prompt appeared to make the graph *cost* 10%. The benchmark flag excludes the chat template's ~8 tokens, so the request arrived at 1032, overshot the 1024 capture bucket and padded to 1280. Re-run so the server sees 1024, the cell reads −1.30%. The capture ladder steps by 16–64 tokens below 1024 and by 256 above, so a request a handful of tokens past a boundary measures padding rather than the graph. Checked across every workload in this report: all land within 0–6.7%, and the image cells land on 1024 exactly.

Limits

- **Resolution floor 3.60%** on the sweep, from repeating the reference cell. Every workload at 720 tokens and above sits inside it; the apparent wobble there is noise, and no sign can be claimed for those cells.
- **The gap-filling workloads were a separate bracket** without a drift cell of their own, so they inherit the main sweep's estimate — a weaker guarantee.
- **Everything is concurrency 1.** Issue #4's batched case is untested, and the graph's value under batching is a different question.
- **All graph results are the breakable backend.** The piecewise backend is absent for the reason given in section 4.
- **Absolute latencies are not production numbers** — the library stack is older than upstream pins. Internal contrasts are unaffected.